

Bank Interest Demonstration

Title: Bank Interest Rate Demonstration using Dynamic Method Dispatch

Objective/Aim:

To demonstrate method overriding and runtime polymorphism in Java by calculating different rates of interest for different banks[cite: 29].

System Requirements :

Hardware

- Standard PC/Laptop[cite: 1]
- Intel Core i3/i5 processor or equivalent[cite: 1]
- Minimum 4 GB / 8 GB RAM[cite: 1]
- Minimum 500 MB free storage[cite: 1]
- Keyboard and Monitor[cite: 1]

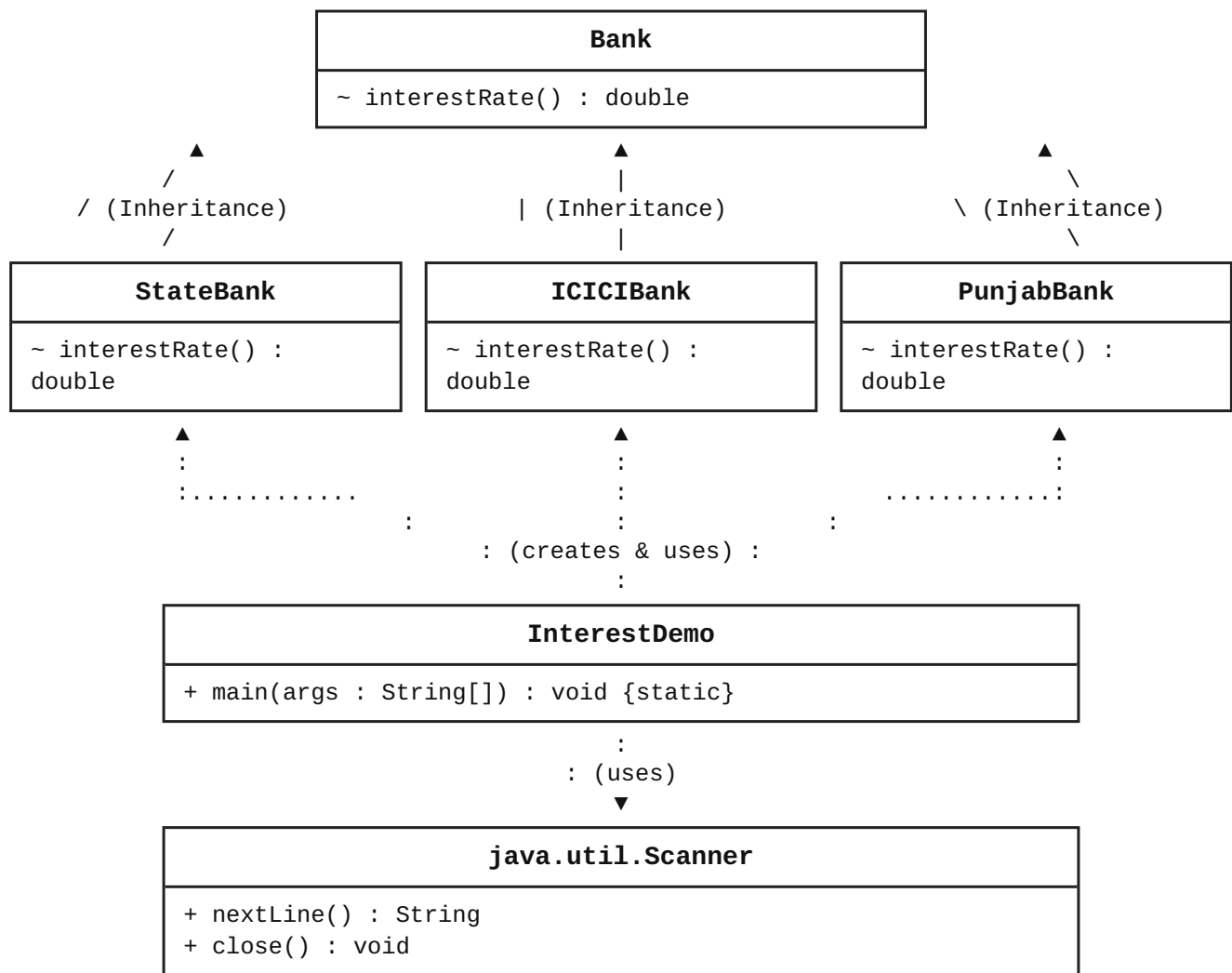
Software

- Operating System: Windows/Linux/macOS[cite: 1]
- Java Development Kit (JDK 17 or JDK 21)[cite: 1]
- Text Editor or IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code[cite: 1, 29]
- Command Prompt/Terminal[cite: 1]

Algorithm / Logic Description:

1. Create a Bank class with an `interestRate ( )` method returning the base interest rate[cite: 29].
2. Create `StateBank`, `ICICIBank`, and `PunjabBank` classes by extending `Bank`[cite: 29].
3. Override the `interestRate ( )` method in each child class with its respective rate[cite: 29].
4. Read the bank name from the user using `Scanner`[cite: 29].
5. Create a `Bank` reference and assign the corresponding child-class object based on the entered bank name[cite: 29].
6. Call `interestRate ( )` through the `Bank` reference to demonstrate runtime polymorphism[cite: 29].
7. Display the selected bank's rate of interest[cite: 29].
8. Stop the program.

UML class Diagram



Relationship: StateBank, ICICIBank, and PunjabBank inherit from Bank and override `interestRate()` [cite: 29]. InterestDemo instantiates them dynamically and uses Scanner for user input [cite: 29].

Source Code:

```
package core_java;

import java.util.Scanner;

class Bank {

    double interestRate() {
        return 4.0;
    }
}

class StateBank extends Bank {

    @Override
    double interestRate() {
```

```

        return 7.0;
    }
}

class ICICIBank extends Bank {

    @Override
    double interestRate() {
        return 6.5;
    }
}

class PunjabBank extends Bank {

    @Override
    double interestRate() {
        return 6.0;
    }
}

public class InterestDemo {

    public static void main(String[] args) {

        Scanner input = new Scanner(System.in);

        System.out.print("Enter bank name: ");
        String name = input.nextLine();

        Bank bankObj;

        if (name.equalsIgnoreCase("RBI")) {

            bankObj = new Bank();

        } else if (name.equalsIgnoreCase("SBI")) {

            bankObj = new StateBank();

        } else if (name.equalsIgnoreCase("ICICI")) {

            bankObj = new ICICIBank();

        } else if (name.equalsIgnoreCase("PNB")) {

            bankObj = new PunjabBank();

        } else {

            System.out.println("Bank name is not available.");
            input.close();
            return;
        }

        double rate = bankObj.interestRate();

        System.out.println("Rate of Interest: " + rate + "%");

        input.close();
    }
}

```

```
}  
}
```

Compilation & Execution Commands:

Save file as: InterestDemo.java[cite: 30]

Compilation: javac -d . InterestDemo.java (or javac InterestDemo.java)[cite: 30]

Execution: java core_java.InterestDemo (or java InterestDemo)[cite: 30]

Output Screenshot:



```
● PS C:\Users\mmeen\RAVI> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+S  
howCodeDetailsInExceptionMessages' '-cp' 'C:\Users\mmeen\AppData\Roaming\Code\User\workspaceStorage\b  
53c69995fa1d8f837d3efeb869e4899\redhat.java\jdt_ws\RAVI_49fdb1a2\bin' 'InterestDemo'  
Enter bank name: SBI  
Rate of Interest: 7.0%  
○ PS C:\Users\mmeen\RAVI> 
```

Result:

The program was implemented successfully to demonstrate method overriding and runtime polymorphism for different banks[cite: 31]. The execution confirms dynamic method dispatch where the appropriate `interestRate()` method is selected and invoked at runtime based on user selection[cite: 29].